

Running the μ OS++ xPack Test Harness on the AArch32 and AArch64 Ports

- [Running the \$\mu\$ OS++ xPack Test Harness on the AArch32 and AArch64 Ports](#)
 - [A step-by-step handbook for the `Work-smp` workspace](#)
 - [1. What you end up with](#)
 - [2. The two test systems, in plain English](#)
 - [3. Prerequisites](#)
 - [4. The target layout](#)
 - [5. Step 1 — Create the workspace and clone the three repositories](#)
 - [6. Step 2 — Create the symlinks](#)
 - [7. Step 3 — Bring the xPack harness into the kernel clone](#)
 - [8. Step 4 — Point the harness at the AArch32 port](#)
 - [9. Step 5 — Keep the C++ standard at 20](#)
 - [10. Step 6 — Create the platform](#)
 - [11. Step 7 — Edit `tests/package.json`](#)
 - [12. Step 8 — Give the kernel a `package.json`](#)
 - [13. Why the semihosting implementation matters](#)
 - [14. Step 9 — Install, prepare, build, test](#)
 - [15. Step 10 — Hardware \(preview\)](#)
 - [16. Doing the same for AArch64](#)
 - [17. Adding a port test to the harness \(`smp-pipeline-test`\)](#)
 - [18. Troubleshooting — every problem encountered, and its fix](#)
 - [19. Repeating this for another test, board or architecture](#)
 - [20. File reference](#)
 - [21. Glossary](#)

Running the μ OS++ xPack Test Harness on the AArch32 and AArch64 Ports

A step-by-step handbook for the `Work-smp` workspace

This document records, in plain English and in order, every step that was taken to make the **xPack test harness** (the `tests/` folder of `micro-os-plus-iii`) build and run the **SMP kernel** through the **AArch32 and AArch64 ports** on the **Raspberry Pi Zero 2 W**, under QEMU and (later) on real hardware.

It is written so that the same procedure can be repeated for the next test, the next board (rpi3b, Luckfox Lyra), or another architecture, without re-discovering any of it.

1. What you end up with

A workspace at `~/Work-smp/micro-os-plus-iii/` in which:

- the **kernel is the SMP implementation** (`micro-os-plus-iii-smp`), cloned under the name `micro-os-plus-iii.git`;
- the **drivers** live in `micro-os-plus-iii-devices.git`;
- the **AArch32 port** lives in `micro-os-plus-iii-aarch32.git`, with its own per-board test/ tree (board code and board tests);
- the **xPack harness** lives in `micro-os-plus-iii.git/tests/` and is driven with `xpm`, exactly like the upstream `README-DEVELOPER.md` workflow;
- a single command builds and runs a test:

```
xpm run test --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

and CTest reports a real **Passed / Failed**, because the test exits through the standard ARM semihosting mechanism.

2. The two test systems, in plain English

Before touching anything, it helps to know that there are **two different test systems** involved, and this guide makes them work together.

2.1 The xPack harness (`tests/`)

This is the system documented in `TESTS-XPack-SYSTEM.md`. It is a *matrix runner*:

- **Test sources** are platform-independent: `tests/sources/<suite>/` — for example `mutex-stress`, `rtos-apis`, `cmsis-os-validator`.
- **Platforms** are folders under `tests/platforms/<name>/` that describe how to build and run those sources on a particular target.
- **Build configurations** in `tests/package.json` name a (platform × toolchain × build type) combination, e.g. `aarch32-rpi-zero-2w-cmake-gcc-debug`.
- The actual build is CMake + Ninja; the actual run is CTest.
- A test passes when its process exits with code 0.

In the original upstream project the harness tests the **plain** kernel. Here we point it at the **SMP kernel, through the AArch32 port**.

2.2 The SMP family and the AArch32 port

- `micro-os-plus-iii-smp` is the RTOS kernel. It exports the same CMake target name as the plain kernel, `micro-os-plus::iii`, but with SMP support.
- `micro-os-plus-iii-devices` holds the drivers (SD, FatFs, DWC2 USB) and the BCM2837 SoC support.

- `micro-os-plus-iii-aarch32` is the **port**: startup, MMU, exception handlers, the SMP scheduler half, and the per-board support. It exports `micro-os-plus::aarch32`, and it **adds the kernel and the devices itself**.

The last point is the single most important rule:

The AArch32 port is the entry point. It adds the kernel and the devices. Never add the kernel separately, or CMake will complain that `micro-os-plus::iii` is defined twice.

3. Prerequisites

- **xpm** (the xPack package manager) and **Node.js ≥ 20**.
- **arm-none-eabi-gcc 15.2.1** (xPack) and **aarch64-none-elf-gcc 15.2.1** (xPack, only to build the QEMU boot shim), both already in `~/.local/xPacks/@xpack-dev-tools/`.
- **qemu-arm 9.x** (xPack) providing `qemu-system-aarch64`.
- The **bare git repositories** of the SMP family, in `/home/dan/Downloads/GIT/`.
- The **harness** to copy, in `~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests/`.

4. The target layout

```
~/Work-smp/micro-os-plus-iii/
├─ micro-os-plus-iii.git/           # the SMP kernel (cloned from -smp, renamed)
│   ├── package.json               # added by hand (see Step 8)
│   ├── cmake/ include/ port/ src/ test_smpl/
│   └─ tests/                      # the xPack harness (see Step 3)
│       ├── package.json           # the matrix (edited, Step 7)
│       ├── cmake/
│       ├── sources/               # mutex-stress, rtos-apis, cmsis-os-validator
│       └─ platforms/
│           └─ aarch32-rpi-zero-2w/ # the new platform (Step 6)
├─ micro-os-plus-iii-devices.git/   # drivers
├─ micro-os-plus-iii-aarch32.git/   # the port, with test/boards/rpi-zero-2w/
├─ micro-os-plus-iii-smp -> micro-os-plus-iii.git
└─ micro-os-plus-iii-devices -> micro-os-plus-iii-devices.git
```

The two symlinks exist because the port looks for its dependencies as siblings named `micro-os-plus-iii-smp` and `micro-os-plus-iii-devices`, but we cloned them with a `.git` suffix (to mirror the Work layout).

5. Step 1 — Create the workspace and clone the three repositories

```
mkdir -p ~/Work-smp/micro-os-plus-iii
cd ~/Work-smp/micro-os-plus-iii

git clone /home/dan/Downloads/GIT/micro-os-plus-iii-smp.git    micro-os-plus-iii.git
```

```
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-devices.git micro-os-plus-iii-devices.git
git clone /home/dan/Downloads/luckfox_lyra/TMP7/micro-os-plus-iii-aarch32 \
micro-os-plus-iii-aarch32.git
```

Why these sources:

- The kernel comes from the **bare SMP repo**, so it is the SMP implementation.
- The port comes from the **TMP7 working copy** because that is where the newest layout lives: the board code and the board tests have moved under test/ (test/boards/<board>/, test/<board>/). The bare aarch32 repo is behind that refactor.

At this point the kernel is named micro-os-plus-iii.git, but it **is** the SMP kernel. Verify:

```
git -C micro-os-plus-iii.git remote -v          # origin = ../micro-os-plus-iii-smp.git
grep -rL OS_USE_SMP_SCHEDULER micro-os-plus-iii.git/include | head -1
```

6. Step 2 — Create the symlinks

```
cd ~/Work-smp/micro-os-plus-iii
ln -s micro-os-plus-iii.git      micro-os-plus-iii-smp
ln -s micro-os-plus-iii-devices.git micro-os-plus-iii-devices
```

The port's CMakeLists defaults are:

```
set (UOS_SMP_DIR      "${uos_siblings}/micro-os-plus-iii-smp"      ...)
set (UOS_DEVICES_DIR "${uos_siblings}/micro-os-plus-iii-devices" ...)
```

The symlinks make those defaults resolve to our clones, so no -D flags are needed. Verify:

```
readlink -f micro-os-plus-iii-smp      # ../micro-os-plus-iii.git
readlink -f micro-os-plus-iii-devices  # ../micro-os-plus-iii-devices.git
```

7. Step 3 — Bring the xPack harness into the kernel clone

The harness is tracked in the plain repository. Copy only the tracked files (so no build/, node_modules/ or xpacks/):

```
git -C ~/Work/micro-os-plus-iii/micro-os-plus-iii.git archive HEAD tests \
| tar -x -C ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git
```

Result: micro-os-plus-iii.git/tests/ with cmake/, sources/, platforms/, package.json, and so on.

The harness is only the *framework*. It contains no kernel, so copying it from the plain repository does not pull in the plain kernel. What the tests link is decided later, in Step 4, when we point the harness at the port.

8. Step 4 — Point the harness at the AArch32 port

The harness has exactly one line that decides “the library under test”. It is in `tests/cmake/tests-main.cmake`:

```
add_subdirectory (".." "top-bin")
```

We replace it with the port:

```
set (UOS_AARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git"
    CACHE PATH "µOS++ III AArch32 port working copy")
if (NOT EXISTS "${UOS_AARCH32_DIR}/CMakeLists.txt")
    message (FATAL_ERROR "Cannot find the AArch32 port at ${UOS_AARCH32_DIR}")
endif ()
add_subdirectory ("${UOS_AARCH32_DIR}" "port-bin")
```

Why this works:

- `tests/` is at `micro-os-plus-iii.git/tests`, so `CMAKE_SOURCE_DIR` is the `tests/` directory, and `../../micro-os-plus-iii-aarch32.git` is the port clone beside the kernel.
- Adding the port adds the SMP kernel (`micro-os-plus::iii`) and the devices, exactly once.
- The port’s own `test/` tree is **not** added, because the port is not the top-level project here (`PROJECT_IS_TOP_LEVEL` is false).

`tests-main.cmake` carries a “DO NOT EDIT! Automatically generated” banner. This working area deliberately overrides it. If the harness is ever regenerated from the build-helper templates, this edit must be re-applied.

9. Step 5 — Keep the C++ standard at 20

The harness builds with **C++20**. In `tests/CMakeLists.txt`:

```
set (CMAKE_CXX_STANDARD 20)
```

The port *sources* build cleanly at C++20, and the harness uses C++20 (as upstream `micro-os-plus-iii` does). The port’s own `uos_add_app()` was also switched from C++23 to C++20. Both AArch32 and AArch64 pass at C++20.

10. Step 6 — Create the platform

Create `tests/platforms/aarch32-rpi-zero-2w/` with six files. Each is explained below.

10.1 `cmake/definitions.cmake`

Names the platform and guards the toolchain version.

```

set (xpack_platform_compile_definition
    "MICRO_OS_PLUS_PLATFORM_AARCH32_RPI_ZERO_2W")

# Reproducibility guard: refuse anything that is not the pinned 15.2 toolchain.
if (DEFINED CMAKE_C_COMPILER_VERSION
    AND NOT CMAKE_C_COMPILER_VERSION MATCHES "^15\\.2\\.")
    message (FATAL_ERROR
        "This platform must be built with the pinned xPack arm-none-eabi-gcc 15.2, "
        "but found ${CMAKE_C_COMPILER} ${CMAKE_C_COMPILER_VERSION}.")
endif ()

```

10.2 cmake/dependencies-folders.cmake

Lists the test sources to add. **Do not list the port here** — it is already added by tests-main.cmake.

```

set (
    xpack_dependencies_folders
    "${CMAKE_SOURCE_DIR}/sources/mutex-stress"
    "${CMAKE_SOURCE_DIR}/sources/rtos-apis"
    "${CMAKE_SOURCE_DIR}/sources/cmsis-os-validator"
    "${CMAKE_SOURCE_DIR}/xpacks/@xpacks/arm-cmsis-rtos-validator"
)

```

10.3 cmake/platform-library.cmake

Creates **two** interface libraries (see §17.2 for why the split is needed):

- micro-os-plus::platform — the **base**: include dirs, flags, the linker script, the port, and (AArch32) the kernel's C-library groups. Harness suites and port tests both link it.
- micro-os-plus::platform-support — adds src/platform-support.cpp (startup hooks + strong main()), for the **harness suites only**.

```

add_library (platform-aarch32-rpi-zero-2w-interface INTERFACE EXCLUDE_FROM_ALL)

target_include_directories (platform-aarch32-rpi-zero-2w-interface INTERFACE "include")

target_compile_definitions (
    platform-aarch32-rpi-zero-2w-interface
    INTERFACE
        "${xpack_platform_compile_definition}"
        OS_NCPU=${UOS_BOARD_NCPU}
        OS_USE_SMP_SCHEDULER=1
        TRACE_SEMIHOST QEMU_BUILD __ARM_EABI__ __ARM_ARCH_7A__
        OS_USE_TRACE_SEMIHOSTING_STDOUT
        _GNU_SOURCE
)

target_compile_options (
    platform-aarch32-rpi-zero-2w-interface
    INTERFACE
        $<$<COMPILE_LANGUAGE:C>:-std=gnull>
        $<$<COMPILE_LANGUAGE:CXX>:-fno-exceptions>
        $<$<COMPILE_LANGUAGE:CXX>:-fno-rtti>
        $<$<COMPILE_LANGUAGE:CXX>:-fabi-version=0>
)

```

```

target_link_options (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE -nostartfiles -Wl,--gc-sections "-T${UOS_BOARD_LINKER_QEMU}"
)

target_link_libraries (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE micro-os-plus::aarch32
             micro-os-plus::iii-newlib-reent
             micro-os-plus::iii-semihosting
)

add_library (micro-os-plus::platform ALIAS platform-aarch32-rpi-zero-2w-interface)

# Support (hooks + main) for the harness suites; port tests link the base only.
add_library (platform-aarch32-rpi-zero-2w-support-interface INTERFACE
  EXCLUDE_FROM_ALL)
target_sources (platform-aarch32-rpi-zero-2w-support-interface
  INTERFACE "src/platform-support.cpp")
target_link_libraries (platform-aarch32-rpi-zero-2w-support-interface
  INTERFACE platform-aarch32-rpi-zero-2w-interface)
add_library (micro-os-plus::platform-support
  ALIAS platform-aarch32-rpi-zero-2w-support-interface)

```

Key points:

- UOS_BOARD_NCPU and UOS_BOARD_LINKER_QEMU are set and cached by the port's board file (test/boards/rpi-zero-2w/board.cmake), so the platform can read them.
- The port's interface already carries the CPU flags (-mcpu=cortex-a53 -marm -mfloat-abi=hard -mfpu=neon-fp-armv8) **both at compile and at link time**, and the board defines (SOC_BCM2837, SEMIHOST_TRAP_HLT, LED_PIN=29, ...). The platform adds only the bare-metal extras and the linker script.
- QEMU_BUILD makes the port select its QEMU semihosting (SVC + reason-by-value); without it the port would use the hardware trap and QEMU would return 1.
- micro-os-plus::iii-newlib-reent and micro-os-plus::iii-semihosting are the kernel's optional groups that provide the C library and the exit. They live in the **base** because port tests use printf too. See section 13.
- The startup hooks and main() are in the **support**, so the port tests (which bring their own strong main/hooks) link the base and not the support.

10.4 CMakeLists.txt

Builds the test executables and registers them with CTest. It also builds the AArch64→AArch32 QEMU boot shim.

The important part:

```

add_test (
  NAME "${PLATFORM_NAME}-mutex-stress-test"
  COMMAND
    qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
    --semihosting-config enable=on,target=native
    --kernel "${_shim_img}"
    --device "loader,file=${<TARGET_FILE:mutex-stress-test>.bin,addr=0x10000"

```

```
)
set_tests_properties ("${PLATFORM_NAME}-mutex-stress-test" PROPERTIES
    TIMEOUT 120)
```

and the raw-image step, needed because the shim and the boot card consume a .bin, not an ELF:

```
add_custom_command (TARGET ${name} POST_BUILD
    COMMAND ${CMAKE_OBJCOPY} -O binary "${<TARGET_FILE:${name}>}" "${<TARGET_FILE:${name}>.bin"
    COMMAND ${CMAKE_SIZE} --format=berkeley "${<TARGET_FILE:${name}>}"
    VERBATIM)
```

10.5 include/cmsis-plus/platform.h

The harness's os-app-config.h includes <cmsis-plus/platform.h>; the port does not provide one, so the platform does. It also enables the kernel's semihosting syscalls and provides the one CMSIS intrinsic the port lacks.

```
#define OS_USE_SEMIHOSTING_SYSCALLS

#if defined(__arm__) && !defined(__disable_irq)
static inline void __disable_irq (void) { __asm__ volatile ("cpsid i" ::: "memory"); }
static inline void __enable_irq (void) { __asm__ volatile ("cpsie i" ::: "memory"); }
#endif
```

10.6 src/platform-support.cpp

The port's own tests each define their startup hooks and their own main(). The harness tests do not, so the platform provides them, mirroring the port's tests:

- os_startup_initialize_hardware_early() — empty.
- os_startup_initialize_hardware() — initialise the UART, the free store, the exception handlers.
- a **strong** main() — set the interrupts stack (the kernel's weak main does not, which leaves a 0-byte IRQ stack and hangs the scheduler), create the main thread, start the scheduler.
- the trampoline calls os_startup_initialize_args() and then os_main(argc, argv), honouring the harness contract.

The full file:

```
#include <cmsis-plus/rtos/os.h>
#include <cstdint>
#include <stdlib>
#include <uart.hpp>
#include <exception_handler.hpp>

extern "C"
{
    extern char __heap_start[], __heap_end[], __fiq_stack_top[], __irq_stack_top[];
    extern void os_startup_create_thread_idle (void);
    extern os::rtos::thread* os_main_thread;

    void os_startup_initialize_hardware_early (void) { }
```

```

void os_startup_initialize_hardware (void)
{
    uart::uart1.init ();
    os_startup_initialize_free_store (
        __heap_start, static_cast<std::size_t> (__heap_end - __heap_start));
    exception::init ();
}

[[noreturn]] static void harness_main_trampoline (void)
{
    int argc = 0;
    char** argv = nullptr;
    os_startup_initialize_args (&argc, &argv);
    int code = os_main (argc, argv);
    std::exit (code);
}

int main (int, char*[])
{
    using namespace os::rtos;
    #if defined(OS_HAS_INTERRUPTS_STACK)
        interrupts::stack ()->set (
            reinterpret_cast<thread::stack::element_t*> (__fiq_stack_top),
            __irq_stack_top - __fiq_stack_top);
        interrupts::stack ()->initialize ();
    #endif
    scheduler::initialize ();

    static thread::stack::element_t main_stack[8192];
    thread::attributes attr = thread::initializer;
    attr.th_stack_address = main_stack;
    attr.th_stack_size_bytes = sizeof (main_stack);

    static thread main_thread{
        "main", reinterpret_cast<thread::func_t> (harness_main_trampoline),
        nullptr, attr
    };
    os_main_thread = &main_thread;

    os_startup_create_thread_idle ();
    scheduler::start ();
    return 0;
}

```

11. Step 7 — Edit tests/package.json

This is the harness matrix. Four things are added or changed.

11.1 A hidden block for the tools

```

"aarch32-actions": {
    "hidden": true,
    "actions": {
        "install": [ "xpm install --config {{ configuration.name }}" ]
    }
}

```

```

    }
  },
  "aarch32-dependencies": {
    "hidden": true,
    "devDependencies": {
      "@xpack-dev-tools/arm-none-eabi-gcc": "15.2.1-1.1.1",
      "@xpack-dev-tools/qemu-arm": "9.2.4-1.1"
    }
  }
}

```

11.2 The build configurations

```

"aarch32-rpi-zero-2w-cmake-gcc-debug": {
  "inherit": [ "aarch32-actions", "cmake-actions",
    "aarch32-dependencies", "short-win-paths-properties" ],
  "properties": {
    "buildType": "Debug",
    "platformName": "aarch32-rpi-zero-2w",
    "toolchainFileName": "arm-none-eabi-gcc.cmake",
    "shortConfigurationName": "a32d"
  }
},
"aarch32-rpi-zero-2w-cmake-gcc-release": {
  "inherit": [ "aarch32-rpi-zero-2w-cmake-gcc-debug" ],
  "properties": { "buildType": "MinSizeRel", "shortConfigurationName": "a32r" }
}

```

11.3 An aggregate action

```

"test-aarch32-rpi-zero-2w-cmake": [
  "xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-release",
  "xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-release",
  "xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-release"
]

```

11.4 Repoint the default and CI actions

install, test, test-all, test-ci and install-ci were changed to use the AArch32 configuration instead of the plain-kernel ones. The old plain configurations (native, qemu-cortex-*, nucleo-*, raspberrypi-pico) are left defined but unreferenced; they would fail against the SMP kernel because they expect micro-os-plus::iii-cortexm / iii-posix-arch.

12. Step 8 — Give the kernel a `package.json`

The harness prints a greeting by reading `../package.json` relative to `tests/`. The SMP kernel repo has no `package.json`, so add a minimal one at `micro-os-plus-iii.git/package.json`:

```
{
  "name": "@micro-os-plus/micro-os-plus-iii-smp",
  "version": "7.1.0",
  "description": "A source code library with the portable part of μOS++ III, SMP variant",
  "license": "MIT"
}
```

Without it, cmake stops with file failed to open for reading: ../tests/./package.json.

13. Why the semihosting implementation matters

This is the heart of making CTest work, and it is the part that is easy to get wrong.

13.1 The problem

A CTest test passes only if its process exits with code 0. For a QEMU run, the process exit code comes from the guest's **semihosting** `SYS_EXIT` call.

The AArch32 port has its own semihosting header (`semihosting.hpp`) which uses the instruction `HLT #0xF000`. That is correct for the **OpenOCD aarch64 debug target** used on real hardware, but QEMU's AArch32 `SYS_EXIT` through that form returns exit code **1**, so CTest reports **Failed** even though the test printed everything and finished.

13.2 The correct implementation

The **original** `micro-os-plus-iii` harness does not use `HLT`. It uses the classic AArch32 semihosting trap **SWI 0x123456** (also written `SVC 0x123456`), implemented in the kernel files:

- `src/libc/newlib/c-newlib-reent.cpp` — the newlib reentrant syscalls (`_write_r`, `_read_r`, `_gettimeofday_r`, ...) which call `__posix_*`;
- `src/semihosting/c-syscalls-semihosting.cpp` — the `__posix_*` layer and the **exit procedure** `os_terminate()` → `report_exception()` → **SWI 0x123456**.

In the **plain** kernel these two files are part of the core. In the **SMP** kernel they were made optional CMake groups:

- `micro-os-plus::iii-newlib-reent`
- `micro-os-plus::iii-semihosting`

They are enabled by defining `OS_USE_SEMIHOSTING_SYSCALLS` (which the harness platforms normally do in their `include/cmsis-plus/platform.h`).

13.3 What we did

- Removed the hand-rolled `_Exit()` override and the hand-written newlib syscalls (they used the port's `HLT`, hence exit code 1).
- Linked `micro-os-plus::iii-newlib-reent` and `micro-os-plus::iii-semihosting`.
- Defined `OS_USE_SEMIHOSTING_SYSCALLS` in the platform header.
- Kept a strong `main()` (for the interrupts stack) that calls `os_main(argc, argv)`.

Now the exit path is:

```
os_main returns
-> std::exit(code)
-> kernel exit() -> _Exit() (weak, in the kernel)
-> os_terminate() (strong, from iiii-semihosting)
-> report_exception(ADP_Stopped_ApplicationExit)
-> SWI 0x123456 -> QEMU exits 0 -> CTest "Passed"
```

Do **not** link `micro-os-plus::iiii-trace-semihosting`: the AArch32 port already defines `os::trace::write` in `src/rtos/os-core.cpp`, and linking the kernel's `trace-semihosting` produces a *multiple definition* link error.

13.4 The linking conditions for CTest (common vs architecture code)

For a C++ test to make CTest report **Passed**, the process must exit with code 0, which on these bare-metal targets means the guest must issue a semihosting `SYS_EXIT` carrying the success reason, in the exact form the host (QEMU or OpenOCD) expects. Getting there depends on the right code being linked **strongly** at two layers.

Common code — the kernel (or the harness standing in for it).

- The C library and the exit come from the SMP kernel's *optional* groups:
 - `micro-os-plus::iiii-newlib-reent` — the newlib reentrant syscalls (`_write_r`, `_gettimeofday_r`, ...) which call `__posix_*`;
 - `micro-os-plus::iiii-semihosting` — the `__posix_*` layer and the **strong** `os_terminate()` → `report_exception()` → SWI 0x123456. On the *plain* kernel these two files are part of the core; on the *SMP* kernel they must be linked explicitly.
- The kernel's `_Exit()` is **weak** and calls `os_terminate()`. With `iiii-semihosting` linked, `os_terminate()` is strong and exits through semihosting; without it, the weak default resets or idles and the test **hangs** until the CTest timeout.
- `OS_USE_SEMIHOSTING_SYSCALLS` must be defined (the `qemu-cortex/nucleo` platforms do it in `include/cmsis-plus/platform.h`), or `c-syscalls-semihosting.cpp` compiles to nothing and `_write/_gettimeofday` stay undefined.
- Supporting macros: `_GNU_SOURCE` (for `S_IREAD`) and, on a port with no CMSIS core, a `__disable_irq` shim.
- A **strong** `main()` that sets the interrupts stack is required — the kernel's weak `main()` does not, and the scheduler hangs. Harness suites get it from `platform-support.cpp`; port tests bring their own.

Architecture-dependent code — the port.

- The kernel's semihosting is **AArch32/Cortex-M only** (it uses SWI/BKPT and `r0/r1`). It cannot be linked on AArch64.
- **AArch32**: the port's `include/semihosting.hpp` selects the trap by build variant — `QEMU_BUILD` → SVC 0x123456 with the reason passed **by value** (QEMU), hardware → HLT #0xF000 (Pi) or SVC (Lyra) with the reason passed **by pointer** (OpenOCD). The port adds a **strong** `_Exit()` (`src/semihosting-exit.cpp`) that overrides the kernel's weak one and exits through this trap.
- **AArch64**: the port provides everything itself — the semihosting (HLT #0xF000, two-field { reason, status } exit), the syscalls (`_write`, `_read`, `_close`, `_lseek`, `_fstat`, `_isatty` in `test/boards/rpi-zero-2w/include/uart.hpp` and `src/handlers.cpp`) and a strong `_Exit()`. The

kernel's `iii-semihosting` / `iii-newlib-reent` groups must **not** be linked (AArch32-only); `_gettimeofday` is added by the harness platform.

In one line: link the *strong* semihosting implementation for the target — the kernel's `iii-newlib-reent` + `iii-semihosting` on AArch32, the port's own on AArch64 — and make sure exactly one `strong main()/_Exit()` is present, so `SYS_EXIT` reaches the host with the success reason and the process exits 0.

14. Step 9 — Install, prepare, build, test

The order matters, because the pinned tools live **inside the build folder**.

```
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests

# 1. top-level tools (cmake, ninja, build-helper, validator, chan-fatfs)
npm install
xpm install

# 2. the pinned cross tools for this configuration
xpm run install --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 3. configure (this is where the toolchain file is applied)
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 4. build
xpm run build --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 5. run under QEMU via CTest
xpm run test --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

Or all of it at once (debug + release):

```
xpm run test-aarch32-rpi-zero-2w-cmake -C ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/
tests
```

Why `install` must come before `prepare/build`

`xpm install --config X` installs the configuration's tools into `build/X/xpacks/`, and `xpm` prepends `build/X/xpacks/.bin` to `PATH` while it runs `prepare` and `build`. That is how the bare name `arm-none-eabi-gcc` in the toolchain file resolves to the **pinned xPack 15.2.1**.

Consequences:

- If you delete `build/<config>/`, you delete the pinned tools too. Re-run `xpm run install --config <config>` before `prepare`.
- If you run bare `cmake` instead of `xpm run`, the system `arm-none-eabi-gcc` (e.g. 16.2.0) may be found instead. The guard in `definitions.cmake` stops the build with a clear message in that case.

Expected result

```
1/1 Test #1: aarch32-rpi-zero-2w-mutex-stress-test ... Passed    35.68 sec
100% tests passed, 0 tests failed out of 1
```

with output ending in Interrupts stack: ... bytes used and Hasta la Vista!.

15. Step 10 — Hardware (preview)

On real silicon the session is driven by the port's own runner, which already implements halt/load/resume, __smp_spin zeroing and the CPSR resume mode:

```
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii-aarch32.git
BOARD=rpi-zero-2w test/hw.sh list
BOARD=rpi-zero-2w test/hw.sh mutex-stress
```

Requirements and rules:

- A J-Link (default) or Olimex probe; PROBE=olimax selects the other config.
 - **One test per power cycle.** The run is a load into RAM over whatever the previous test left there; the Pi has no reset a script can drive.
 - Keep your own terminal on the UART; the runner does not open it. The verdict is read from the semihosted console in OpenOCD's log.
 - On hardware, the port's HLT #0xF000 semihosting is the right form (the OpenOCD target is aarch64). The SWI form is only needed under QEMU.
-

16. Doing the same for AArch64

Everything above applies, with a handful of differences. This section lists them and the exact extra files, so the AArch64 port can be set up by analogy.

16.1 Clone and select

```
cd ~/Work-smp/micro-os-plus-iii
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-aarch64.git micro-os-plus-iii-aarch64.git
```

No symlink is needed for the port itself (the harness finds it through UOS_AARCH64_DIR), and the port finds the kernel and devices through the existing micro-os-plus-iii-smp / micro-os-plus-iii-devices symlinks.

tests-main.cmake must choose the port from the platform name:

```
set (UOS_AARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git" CACHE PATH "")
set (UOS_AARCH64_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch64.git" CACHE PATH "")
if (PLATFORM_NAME MATCHES "^aarch32")
    add_subdirectory ("${UOS_AARCH32_DIR}" "port-bin")
elseif (PLATFORM_NAME MATCHES "^aarch64")
    add_subdirectory ("${UOS_AARCH64_DIR}" "port-bin")
```

```

else ()
  add_subdirectory (".." "top-bin")    # plain kernel
endif ()

```

16.2 The AArch64 platform

Create `tests/platforms/aarch64-rpi-zero-2w/` with the same six files as for AArch32. The differences:

- **Toolchain:** `aarch64-none-elf-gcc.cmake`; the guard checks 15.2.
- **CPU flags** come from the port (`-mcpu=cortex-a53`, AArch64); no `-marm`.
- **Defines:** `__ARM_ARCH_8A__` instead of `__ARM_ARCH_7A__`.
- **No boot shim:** QEMU's `raspi3b` starts the cores in AArch64, so the test image is handed to `-kernel` directly:

```

add_test (NAME "${PLATFORM_NAME}-mutex-stress-test" COMMAND
  qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
  --semihosting-config enable=on,target=native
  --kernel "$<TARGET_FILE:mutex-stress-test>.bin")

```

- **Strong semihosting:** the AArch64 port's `semihosting.hpp` uses `HLT #0xF000` with a **two-field** { reason, status } exit block, which QEMU maps to the process exit code. The platform therefore provides a strong `_Exit()` calling `semihosting::exit_success()` / `exit_failure()`. The kernel's AArch32-only `iii-semihosting` groups are **not** linked.
- **The port already provides the other syscalls:** `_write`, `_read`, `_close`, `_lseek`, `_fstat` and `_isatty` are defined in the port's `test/boards/rpi-zero-2w/include/uart.hpp` and `src/handlers.cpp`. So `platform-support.cpp` adds only `_gettimeofday` and `_Exit`. Defining the others would be a *multiple definition* link error.

16.3 package.json

Add the AArch64 analogues of the AArch32 blocks:

```

"aarch64-actions": { "hidden": true, "actions": {
  "install": [ "xpm install --config {{ configuration.name }}" ] } },
"aarch64-dependencies": { "hidden": true, "devDependencies": {
  "@xpack-dev-tools/aarch64-none-elf-gcc": "15.2.1-1.1.1",
  "@xpack-dev-tools/qemu-arm": "9.2.4-1.1" } }

```

plus the two configurations (`aarch64-rpi-zero-2w-cmake-gcc-debug` and `-release`, toolchain `aarch64-none-elf-gcc.cmake`) and the `test-aarch64-rpi-zero-2w-cmake` aggregate action.

16.4 Build and run

Exactly as for AArch32:

```

cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run install --config aarch64-rpi-zero-2w-cmake-gcc-debug
xpm run prepare --config aarch64-rpi-zero-2w-cmake-gcc-debug

```

```
xpm run build    --config aarch64-rpi-zero-2w-cmake-gcc-debug
xpm run test     --config aarch64-rpi-zero-2w-cmake-gcc-debug
# or both debug and release:
xpm run test-aarch64-rpi-zero-2w-cmake
```

Result (C++20):

```
1/1 Test #1: aarch64-rpi-zero-2w-mutex-stress-test ... Passed    35.6 sec
100% tests passed, 0 tests failed out of 1
```

16.5 Why AArch64 needs no SWI groups

On AArch32 the port's HLT exit returned QEMU exit code 1 (single-field block), so we linked the kernel's SWI 0x123456 groups. On AArch64 the port's HLT uses the two-field block QEMU expects, so the port's own semihosting is already correct and no kernel group is needed.

17. Adding a port test to the harness (smp-pipeline-test)

So far the harness has built one of its own suites (mutex-stress). The port also ships its **own** tests under test/<board>/, and they can be run through the same CTest system. This chapter adds smp-pipeline-test and explains the one structural change it forces.

17.1 Port test vs harness suite

	harness suite	port test
lives in	tests/sources/<name>/	<port>/test/<board>/<name>/
provides	only the test logic	its own main() and startup hooks
links	micro-os-plus::platform + platform-support	micro-os-plus::platform only
verdict	returns from os_main (exit code)	prints RESULT:, ends via hw_result

The port tests are the ones that have always run on the Pi; the harness merely gives them a CTest face.

17.2 The platform split (do this once)

A port test defines its own main() and os_startup_initialize_hardware*(). The harness's platform-support also defines them. If both are **strong**, the link fails; if both are **weak**, the linker may pick the kernel's weak main (which does *not* set the interrupts stack) and the test hangs. So the platform is split in two:

- micro-os-plus::platform — include dirs, flags, the linker script, the port, and (AArch32) the kernel's iiii-newlib-reent + iiii-semihosting C library. **Port tests link this.**
- micro-os-plus::platform-support — src/platform-support.cpp (startup hooks
 - strong main(), plus AArch64's _Exit/_gettimeofday). **Harness suites link base + support.**

Both platform-library.cmake files already implement this split.

17.3 Make the port's QEMU semihosting exit 0

The port test ends through `hw_result::ok()` → the port's semihosting. Under QEMU that must be the classic trap with the reason passed **by value**, so the harness platform defines `QEMU_BUILD`, and the port's `semihosting.hpp` selects the QEMU form (the TMP7 change). If the port clone predates that change, sync `include/semihosting.hpp` first.

17.4 Add the executable

In `tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt` (AArch64 analogous):

```
option (ENABLE_SMP_PIPELINE_TEST "Build the port's smp-pipeline-test" ON)
if (ENABLE_SMP_PIPELINE_TEST)
  set (_port_test_dir "${UOS_AARCH32_DIR}/test/rpi-zero-2w")
  add_executable (smp-pipeline-test
    "${_port_test_dir}/smp-pipeline-test/main.cpp"
    "${_port_test_dir}/src/test-smp-boot.cpp")
  target_include_directories (smp-pipeline-test PRIVATE "${_port_test_dir}/include")
  add_custom_command (TARGET smp-pipeline-test POST_BUILD
    COMMAND ${CMAKE_OBJCOPY} -O binary "$<TARGET_FILE:smp-pipeline-test>"
    "$<TARGET_FILE:smp-pipeline-test>.bin"
    VERBATIM)
  target_link_libraries (smp-pipeline-test PRIVATE
    micro-os-plus::common-options micro-os-plus::iii
    micro-os-plus::platform          # base only, NOT platform-support
    micro-os-plus::devices)          # the port test reaches the SD card
  # ... SD image + add_test, below
endif ()
```

Key points:

- Link the **base** `micro-os-plus::platform`, never `platform-support`.
- Link `micro-os-plus::devices` only if the test is in the board's `BOARD_TEST_NEED_DEVICES` list (`sd_test smp-mat-sdcard-test smp-num-test smp-pipeline-test usb_test`).
- Include the port test's shared `include/` and compile its `src/test-smp-boot.cpp`.
- The test's own strong `main/hooks` come from its `main.cpp` and override the harness's (which are not linked here).

17.5 The SD image and the CTest command

`smp-pipeline-test` formats a blank SD card with `flatfs`, so the QEMU run needs an SD drive:

```
set (_pipe_disk "${CMAKE_CURRENT_BINARY_DIR}/smp-pipeline-test.disk.img")
add_custom_command (OUTPUT "${_pipe_disk}"
  COMMAND truncate -s 4G "${_pipe_disk}" VERBATIM)
add_custom_target (smp-pipeline-test-disk ALL DEPENDS "${_pipe_disk}")

add_test (NAME "${PLATFORM_NAME}-smp-pipeline-test" COMMAND
  qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
  --semihosting-config enable=on,target=native
  --drive "file=${_pipe_disk},if=sd,format=raw"
  --kernel "${_shim_img}"          # AArch32; AArch64 uses the .bin directly
```

```
--device "loader,file=${TARGET_FILE:smp-pipeline-test}.bin,addr=0x10000")
set_tests_properties ("${PLATFORM_NAME}-smp-pipeline-test" PROPERTIES
    TIMEOUT 1200)
```

Prefix every CTest test name with `${PLATFORM_NAME}`. A CTest test name is what every test runner shows as the leaf label; the *executable* name can stay short. Because `add_test(NAME "mutex-stress-test")` is identical in the aarch32 and aarch64 platforms, a runner (VS Code's Test Explorer, the xPack actions view, a flat `ctest -N`) cannot tell them apart. Prefixing gives `aarch32-rpi-zero-2w-mutex-stress-test` and `aarch64-rpi-zero-2w-mutex-stress-test`. `ctest -R <name>` still matches, because it is a substring match.

Do the same for the CMake project name. `tests/CMakeLists.txt` uses `project(micro-os-plus-micro-os-plus-iii-${PLATFORM_NAME}-tests)`, so a tool that groups tests by the CMake project shows two distinct suites (`...-aarch32-rpi-zero-2w-tests` and `...-aarch64-rpi-zero-2w-tests`) instead of one. Without it, both configurations report the same project name.

17.6 Build and run

```
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run build --config aarch32-rpi-zero-2w-cmake-gcc-debug
# all tests, or just the new one:
PATH="$PWD/build/aarch32-rpi-zero-2w-cmake-gcc-debug/xpacks/.bin:$PATH" \
  ctest --test-dir build/aarch32-rpi-zero-2w-cmake-gcc-debug \
    -R smp-pipeline-test -V
```

Result:

```
1/3 Test #1: aarch32-rpi-zero-2w-mutex-stress-test ... Passed    35.8 sec
2/3 Test #2: aarch32-rpi-zero-2w-smp-pipeline-test ... Passed    33.9 sec
3/3 Test #3: aarch32-rpi-zero-2w-smp-pro-cons-test ... Passed     4.8 sec
100% tests passed, 0 tests failed out of 3
```

17.7 Adding the other port tests

The same recipe applies to `sd_test`, `smp-num-test`, `smp-mat-sdcard-test`, `smp-mat-test`, `smp-pro-cons-test`, `smp_test0..4`, and `usb_test`. The only variables are: whether the test needs `micro-os-plus::devices` (the `BOARD_TEST_NEED_DEVICES` list) and whether it needs an SD drive. `usb_test` additionally needs a host-side driver while it runs, so it is not suitable for an unattended CTest run.

Checklist:

- ☐ the port's `semihosting.hpp` has the `QEMU_BUILD` path (or sync it)
- ☐ the platform is split into base + support (done)
- ☐ executable from the port test source + `src/test-smp-boot.cpp`
- ☐ include the port test's `include/`
- ☐

- ☐ link micro-os-plus::platform (base) + devices if needed
- ☐ .bin step
- ☐ SD image + --drive if the test touches the card
- ☐ add_test with a generous TIMEOUT
- ☐ ctest -R <name> passes

18. Troubleshooting — every problem encountered, and its fix

Symptom	Cause	Fix
file failed to open ... tests/.../package.json	the SMP kernel has no package.json	add a minimal one at the kernel root (Step 8)
CMake error: alias micro-os-plus::iii already exists	the kernel was added twice	only add the port; never add the kernel separately
CMake error: source dir already used	the port re-adds the kernel you also added	add only the port
Configure used /usr/bin/cc / arm-none-eabi-gcc 16.2.0	build run without prepare, or build dir deleted	run install then prepare then build; the guard enforces 15.2
undefined reference to os_startup_initialize_hardware_early/hardware	the harness test does not provide the startup hooks	platform-support.cpp provides them
undefined reference to _write/_read/_gettimeofday/...	the SMP kernel keeps the C library in optional groups	link micro-os-plus::iii-newlib-reent + micro-os-plus::iii-semihosting
undefined reference to os_startup_initialize_args	OS_USE_SEMIHOSTING_SYSCALLS not defined	define it in the platform header
'S_IREAD' was not declared	GNU extension not enabled	add _GNU_SOURCE
'__disable_irq' was not declared	the port has no CMSIS core headers	define it in the platform header
multiple definition of os::trace::write	linked both the port's trace and iii-trace-semihosting	do not link iii-trace-semihosting
Test prints everything then ***Failed, QEMU exit 1	exit via the port's HLT semihosting	use the kernel's SWI 0x123456 exit (section 13)
Test hangs after scheduler::start(), Interrupts stack size: 0 bytes	kernel weak main used, interrupts stack not set	provide the strong main() (Step 10.6)
no such file ... xpacks/.bin/arm-none-eabi-gcc after a clean	build dir (with xpacks) was deleted	re-run xpm run install --config <config>
guard still reports the wrong compiler after a failed configure	the build dir cached the compiler before the guard fired	

Symptom	Cause	Fix
		delete build/<config>/, then install → prepare → build
multiple definition of _write/_read/_close/_lseek/_fstat/_isatty (AArch64)	the port already defines them (uart.hpp, handlers.cpp)	do not define them in platform-support.cpp; add only _gettimeofday and _Exit
AArch64 -mcpu/float link error	AArch64 has no -marm/-mfloat-abi; those are AArch32-only	let the port carry the AArch64 flags

19. Repeating this for another test, board or architecture

17.1 Add another test suite

1. Copy the suite to tests/sources/<name>/.
2. Add "\${CMAKE_SOURCE_DIR}/sources/<name>" to the platform's dependencies-folders.cmake.
3. Add an if (ENABLE_<NAME>_TEST) block to the platform CMakeLists.txt that links test::<name> and registers add_test.
4. If the suite needs extra defines or libraries (for example chan-fatfs), add them in that block.

17.2 Add another board

The AArch32 port already knows zero2w, rpi3b and luckfox-lyra through test/boards/<board>/board.cmake. Add a platform per board (or parameterise the platform with -D BOARD=), and a matching build configuration in package.json.

17.3 Add the AArch64 port

Same shape as AArch32 (see §16 for the full walk-through): clone micro-os-plus-iii-aarch64.git, add a platform that links micro-os-plus::aarch64, uses aarch64-none-elf-gcc.cmake, and runs QEMU raspi3b directly (no shim). The AArch64 port provides its **own** semihosting (HLT #0xF000, two-field exit) and syscalls, so the kernel's AArch32-only iii-semihosting / iii-newlib-reent groups must **not** be linked.

17.4 Checklist

- ☐ kernel cloned as micro-os-plus-iii.git, from -smp
- ☐ symlinks micro-os-plus-iii-smp, micro-os-plus-iii-devices
- ☐ harness tests/ copied
- ☐ tests-main.cmake adds the port, not ..
- ☐ CMAKE_CXX_STANDARD 20
- ☐

platform files: definitions, dependencies-folders, platform-library, CMakeLists, platform.h,
platform-support.cpp



package.json: hidden blocks, configs, aggregate, repointed defaults



kernel package.json



toolchain guard



install → prepare → build → test passes

20. File reference

Concern	Path (relative to ~/Work-smp/micro-os-plus-iii/)
Kernel (SMP)	micro-os-plus-iii.git/
Kernel package.json	micro-os-plus-iii.git/package.json
Harness	micro-os-plus-iii.git/tests/
Harness entry edit	micro-os-plus-iii.git/tests/cmake/tests-main.cmake
Harness standard	micro-os-plus-iii.git/tests/CMakeLists.txt
Harness matrix	micro-os-plus-iii.git/tests/package.json
New platform (AArch32)	micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/
New platform (AArch64)	micro-os-plus-iii.git/tests/platforms/aarch64-rpi-zero-2w/
Platform library (base + support)	.../cmake/platform-library.cmake (micro-os-plus::platform, micro-os-plus::platform-support)
Platform definitions/guard	.../cmake/definitions.cmake
Platform dependencies	.../cmake/dependencies-folders.cmake
Platform executables/tests	.../CMakeLists.txt
Platform header	.../include/cmsis-plus/platform.h
Platform support source	.../src/platform-support.cpp (hooks/main; harness suites)
Port test (smp-pipeline)	micro-os-plus-iii-aarch32.git/test/rpi-zero-2w/smp-pipeline-test/
Devices	micro-os-plus-iii-devices.git/
AArch32 port	micro-os-plus-iii-aarch32.git/
AArch64 port	micro-os-plus-iii-aarch64.git/
Board facts	micro-os-plus-iii-aarch32.git/test/boards/rpi-zero-2w/board.cmake

Concern	Path (relative to ~/Work-smp/micro-os-plus-iii/)
Hardware runner	micro-os-plus-iii-aarch32.git/test/hw.sh
Build folder	micro-os-plus-iii.git/tests/build/aarch32-rpi-zero-2w-cmake-gcc-debug/

21. Glossary

Term	Meaning
xpm	The xPack package manager CLI.
xPack	An npm package with an "xpack" section; used for tools and source libraries.
build configuration	A named (platform, toolchain, build type) tuple in package.json.
action	A named command run with xpm run.
hidden block	A configuration fragment meant only for inherit.
port	The architecture-specific project (micro-os-plus-iii-aarch32) that adds the kernel and devices and exports micro-os-plus::aarch32.
semihosting	The ARM mechanism by which the target asks the host (QEMU or a debugger) to do I/O and to exit.
SWI/SVC 0x123456	The classic AArch32 semihosting trap.
HLT #0xF000	The AArch64 (and AArch32-under-aarch64-debugger) semihosting trap.
_Exit	The libc function exit() ends with; the kernel defines it weak, so a platform can replace it.
os_terminate	The kernel's semihosting exit procedure, strong when iii-semihosting is linked.
interrupts stack	A separate stack used while handling interrupts; the AArch32 port must have it set or the scheduler hangs.
shim	A 20-line AArch64 stub that drops QEMU's Cortex-A53 to AArch32 and jumps to the image at 0x10000.
CTest	CMake's test runner; it checks the process exit code.

Companion to TESTS-XPack-SYSTEM.md and AARCH32-RPI-ZERO-2W-TESTS.md. Verified against the master/working branches of the -smp, -devices, -aarch32 and -aarch64 repositories and the tests/ folder of micro-os-plus-iii.